

EVAC1

1 Justification of Representation and Algorithm 479 Words

I chose to represent the problem as a single-objective constrained optimisation task. The primary objective of the task is to minimise the cost of the diet, while satisfying a set of nutritional and portion constraints. A real-valued vector representation was chosen to represent the quantity of each food item in the diet, as this allows for a more precise and flexible representation of the solution space. Each element of the vector corresponds to the quantity of a specific food item, and the values can be adjusted to meet the constraints while minimising cost. A continuous representation is chosen over a binary or discrete one, as this allows the algorithm to search the solution space with the required precision to find an acceptable solution that does not violate any constraints. This real-valued representation also aligns with the provided constraints which specify non integer quantities of food items, such as the requirement for at least 0.5 units of fish. To handle the constraints a penalty function is used to assign a fitness value to each solution based on how well it satisfies the constraints. Instead of using a binary penalty that simply classifies solutions as feasible or infeasible, a continuous penalty function is used to provide a gradient of fitness values that reflect the degree of constraint violation, to create a selective pressure towards feasible solutions.

The algorithm chosen for implementation is the $(\mu + \lambda)$ strategy. This strategy is selected for its elitism, which will allow for stronger solutions to be preserved across generations, which is crucial in a search space with hard constraints. The (μ, λ) strategy, which does not retain parents, is not chosen as it may lead to the loss of high fitness solutions if they are not selected for reproduction in a given generation. By retaining acceptable solutions throughout generations, the algorithm can concentrate its search in a smaller search space primarily filled with feasible solutions, which can lead to faster convergence towards an optimal solution. A hall of fame is employed to ensure that the best solution found during the entire run of the algorithm is preserved and can be returned at the end, even if it is not present in the final population.

For selection, tournament selection is used. The constraint penalties may introduce significant disparities in fitness values, which can lead to premature convergence around suboptimal solutions if a fitness proportional selection method is used. Tournament selection, on the other hand, maintains a constant selection pressure by selecting the best individual from a randomly chosen subset of the population, which helps to avoid stagnation and encourages exploration of the solution space. For crossover, blend crossover is used, and for mutation, Gaussian mutation is used. The food units are continuous values, and these operators are well-suited for handling real-valued representations, allowing for effective exploration and exploitation of the search space. A simple swap operation will not provide the necessary precision to navigate the solution space effectively, and may struggle to find feasible solutions that satisfy the constraints.

2 Investigation of Parameters 499 Words

2.1 Initial Screening

A 3-level grid search evaluated 648 valid parameter combinations across 10 independent random seeds (6,480 total runs). To maintain tractability, several parameters were fixed based on preliminary testing:

- Mutation mean was set to 0, to avoid introducing directional bias
- The penalty multiplier was set to 100, based on preliminary testing that showed this ensures robust validation guidance without dominating fitness tracking
- Lambda and mu were set to 100 and 200 respectively, providing a reasonable selection pressure while maintaining a diverse population.

This is a feasible number of experiments to run, and will provide enough data to identify trends and make informed decisions about which parameters have the most significant impact on the algorithm’s performance. Results can be seen in table 1.

Table 1: Initial Screening: Top 10 Configurations Sorted by Lowest Cost

Tournament Size	Crossover		Mutation			Mean (£)	Std (£)	Best (£)
	α	p_c	σ	p_m	p_{ind}			
5	1.0	0.8	0.20	0.05	0.40	14.908127	8.370606	12.083912
5	1.0	0.4	0.40	0.20	0.40	13.440277	2.158157	12.084434
5	1.0	0.8	0.05	0.20	0.05	12.497021	1.167626	12.085450
7	1.0	0.4	0.40	0.40	0.20	14.405428	5.914491	12.085904
3	1.0	0.8	0.05	0.05	0.20	12.218463	0.174851	12.086120
5	0.5	0.8	0.40	0.05	0.40	14.163480	2.803105	12.087194
3	1.0	0.8	0.05	0.05	0.05	12.116199	0.032527	12.087575
5	1.0	0.8	0.40	0.05	0.05	16.787340	3.988521	12.087600
3	1.0	0.8	0.20	0.05	0.05	12.110973	0.020694	12.087771
5	1.0	0.8	0.05	0.05	0.05	16.456210	9.270274	12.089038

2.2 Investigation of Trends

In the top 10 configurations the substantial variance across seeds suggested structural instability. Initially, I hypothesised that this variance is due to the mutation operator, as it introduces more variance into the population. However, on visual inspection of the results, it seems that tournament size may be the more significant factor. Tournament size directly impacts selection pressure, which can influence the stability of the search process. A Levene’s test comparing tournament sizes of 3 and 5 found no statistically significant difference in variance, with a p-value of 0.8, but a Mann-Whitney U test comparing the mean costs found a significant decrease in mean cost with a tournament size of 5, with a p-value of 0.02.

As the variance is not caused by tournament size I revisited the mutation probability investigation. Contradictory to my initial hypothesis, Levene’s test revealed a decrease in variance with a mutation probability of 0.20, with a p-value of 0.046, and a Mann-Whitney U test found a significant decrease in mean cost with a p-value of 0.02. This rejects my assumption that mutation merely introduces damaging variance, and suggests that the variance introduced by mutation helps to maintain diversity in the population, preventing premature convergence, which likely causes the observed high variance. A higher rate of mutation was not included to maintain the optimum crossover probability of 0.8.

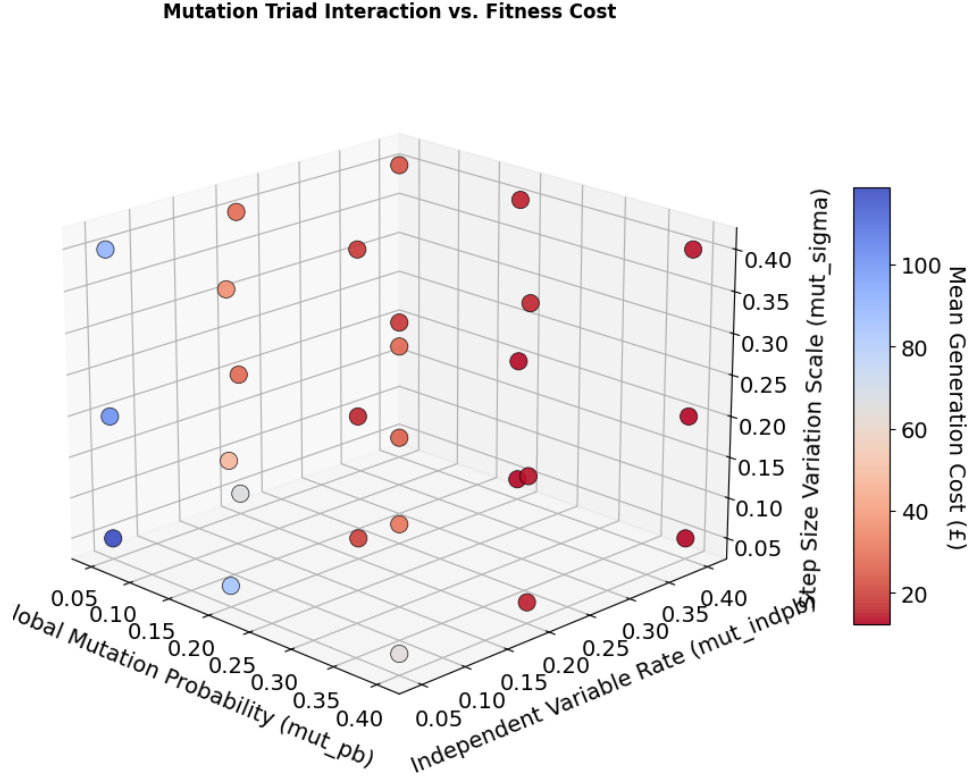


Figure 1: Initial Screening: Mutation Bubble Plot

It is consistent across the top 10 results that a crossover probability of 0.8 performs better than a crossover probability of 0.4, and a crossover alpha of 1.0 performs better than a crossover alpha of 0.5, and is supported with the heatmap in figure 2. For statistical certainty, I performed Mann-Whitney U tests to compare the values found in the top 10 of the initial screening, which found at a p-value of 0.00 that a crossover probability of 0.8 and a crossover alpha of 1.0 both yield significantly lower mean costs than their counterparts.

Finally, a mutation indpb and mutation sigma of 0.40 and 0.20 have been selected as supported by the bubble plot in figure 1, where when mutation probability is 0.20, these values are associated with the lowest mean costs.

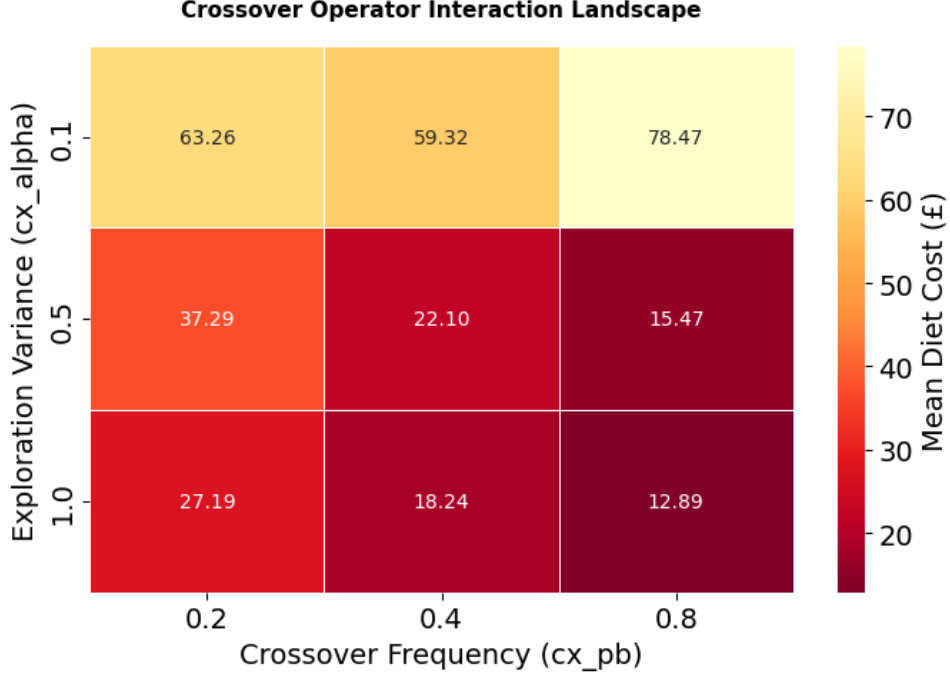


Figure 2: Initial Screening: Crossover Heatmap

3 Evaluation of Solution 351 Words

3.1 Algorithm Robustness

The finalised configuration was evaluated over 500 independent runs to assess the consistency and reliability of the algorithm.

- Minimum Achieved Cost: 12.0865
- Maximum Achieved Cost: 54.7035
- Global Mean Cost: 12.6985
- Standard Deviation: 3.744768

These metrics show that the algorithm can converge to a very strong solution, and the close proximity of the global mean to the absolute minimum confirms the algorithm successfully converges in the vast majority of trials. However, the substantial maximum outlier and standard deviation indicate that the search profile remains vulnerable to stochastic volatility, and occasionally the algorithm fails to escape sub-optimal areas of the parameter space. While the hyperparameter tuning yields good performance, when a larger number of trials are ran, it seems that the algorithm can still struggle to find an acceptable solution.

3.2 Sensitivity to Constraint Changes

To analyse the algorithm’s sensitivity to constraints, I conducted a series of experiments where conducted where the constraints were systematically relaxed and tightened. For each constraint I modified the constraint threshold by $\pm 10\%$ and observed the impact on the algorithm’s performance, over 30 independent runs. Results for this can be seen in figure 3. When bounds were relaxed, the minimum achieved cost falls from 12.08 to 9.88. This makes sense, each constraint only changes a small amount, but when compounded with other relaxed constraints there is a larger benefit. Conversely, tightening the constraints caused the mean cost to increase more

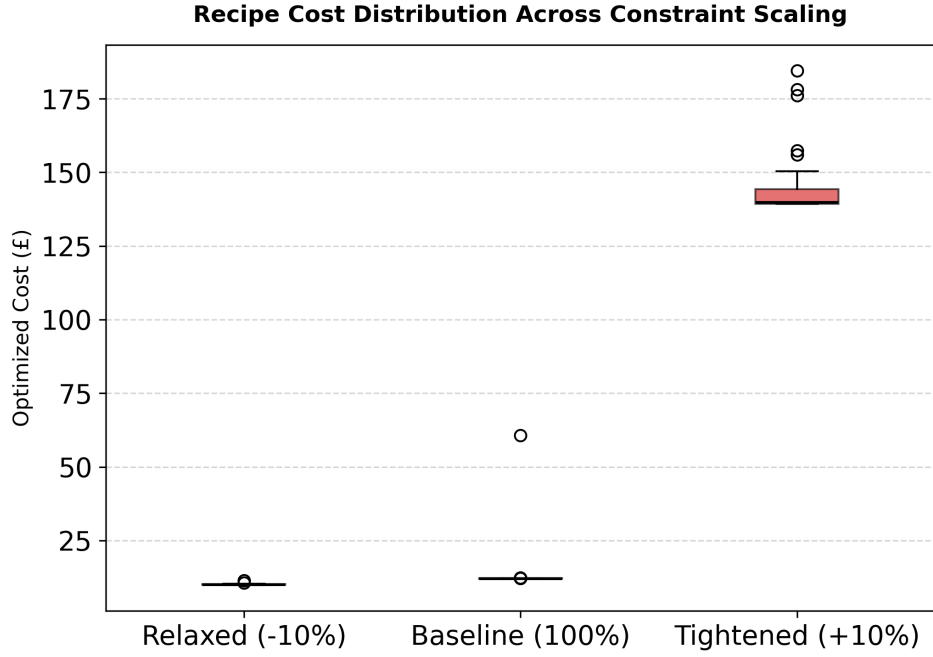


Figure 3: Constraint Sensitivity Analysis

than 10 fold. This extreme cost surge is a structural indicator that the tightened environment makes finding a valid solution exponentially harder. Based on the penalty function and cost of the items, we can also confirm with a final cost of 145, the solution did not converge on a valid solution. In this tightened environment the algorithm cannot find a valid solution and the population has a greater number of outliers than in the other two tests. The final algorithm is not sensitive to constraint relaxing, but when the constraints are tightened it cannot find an acceptable solution.